L0: Reliable AI Streams

Dr. Laszlo Attila Vekony

Modern LLM applications increasingly depend on *streaming* responses: chat UIs, agent runtimes, tool calls, real-time summarization, and multimodal generation. But today's provider streams are not a deterministic substrate. They are best-effort event feeds with failure modes that make production reliability, auditability, and reproducibility expensive and fragile.

L0 (`reliable-ai-streams`) is a deterministic streaming execution substrate that wraps existing model streams and upgrades them into a contract you can build systems on. It provides token-level normalization, smart retries with error-category-aware backoff, streaming guardrails, drift detection, checkpoint-based resumption, model fallbacks, multi-model consensus, structured output validation, streaming pipelines, event sourcing with byte-for-byte replay, and built-in telemetry with OpenTelemetry and Sentry integrations.

L0 is provider-agnostic. Built-in adapters support Vercel AI SDK, OpenAI, Anthropic, and Mastra, with an extensible adapter registry for custom providers. It handles text, structured JSON, and multimodal streams (image, audio, video) under the same deterministic contract.

Available in TypeScript (`npm install reliable-ai-streams`) and Python (`uv add reliable-ai-streams-py`) with full lifecycle and event signature parity.

—

A reliability + observability layer for token streams

> LLMs produce high-value reasoning over a low-integrity transport layer. Streams stall, drop tokens, reorder events, violate timing guarantees, and expose no deterministic contract. L0 fixes the transport so you can build reliable systems on top of any AI stream.

LLM streaming | reliability | deterministic replay | guardrails | drift detection | event sourcing | observability

The Problem: High-Value Reasoning on a Low-Integrity Transport

Streaming is where most production LLM failures actually happen. Even if a model is "fine," the stream can:

- **Stall**: no first token for seconds, or long gaps between tokens (TTFT gaps, inter-token stalls).
- **Disconnect mid-stream**: generation halts unexpectedly, yielding only partial output with no clean recovery path.
- **Reorder or drop chunks**: out-of-order sequences or missing segments, especially under load shedding (429/503).
- **Return empty or near-empty responses**: structurally valid but semantically void payloads that pass naive health checks.
- **Degrade format**: output shifts from structured to broken/ambiguous forms (e.g., Markdown fences left open, JSON braces unmatched, LaTeX environments unclosed).
- **Drift semantically**: unexpected changes in tone, intent, or content mid-generation – repetition loops, meta-commentary, entropy spikes, hedging spirals.
- **Fail silently**: provider-specific behaviors that lack sufficient visibility or hooks for debugging – backgrounded connections, SSE aborts, DNS failures, partial chunks.

The result: retries become guesswork, supervision becomes fuzzy, and reproducibility becomes nearly impossible. Every team building on LLM streams ends up writing ad-hoc retry loops, custom timeout handlers, and fragile format validators. L0 replaces all of that with a single deterministic layer.

—

L0's Thesis

A robust LLM stack needs something analogous to a database's transaction/log layer or a distributed system's consensus/observability foundation:

- **Deterministic lifecycle** – every execution follows the same state machine, regardless of provider.
- **Explicit error taxonomy** – network errors, model errors, content errors, and fatal errors map to distinct recovery behavior.
- **Streaming-safe validation** – guardrails that validate output *as it streams*, not just at the end.

- **Replayable execution** – every token, retry, and decision recorded as an immutable event log.
- **First-class telemetry** – timing, throughput, errors, retries, violations, and drift are built-in outputs, not afterthoughts.
- **Recovery primitives designed for streams** – checkpoint resumption, continuation deduplication, and fallback chains.

L0 treats a model stream as a noisy transport and upgrades it into a deterministic, observable runtime.

—

Design Goals

1. **Determinism by contract**

Every execution follows the same lifecycle and emits a consistent event shape, independent of provider quirks. The lifecycle is specified precisely enough to be ported across languages with identical behavior.

1. **Stream-neutral integration**

You "bring your stream." L0 adapts to Vercel AI SDK, OpenAI, Anthropic, Mastra, and any custom provider via its adapter registry. Text, structured, and multimodal streams all work under the same contract.

1. **Reliability without rewriting meaning**

Guardrails are pure validation functions. They inspect streaming output and signal whether to retry or halt – they never rewrite content. Integrity is preserved, not synthesized.

1. **Observability as a first-class output**

Telemetry is built in: timing, throughput, errors, retries, violations, drift, and network diagnostics. OpenTelemetry and Sentry integrations ship out of the box. Every lifecycle phase fires observable callbacks.

1. **Performance headroom**

The substrate must stay far ahead of model inference speeds. L0 uses incremental state tracking (O(delta) per token), sliding-window drift detection, and tunable check intervals to sustain ~260K tokens/s with full features enabled – orders of magnitude above current and next-generation inference speeds.

1. **Safety-first defaults**

Checkpoint continuation is off by default. Structured objects are never resumed mid-stream. No silent corruption. Every opt-in feature requires explicit enablement.

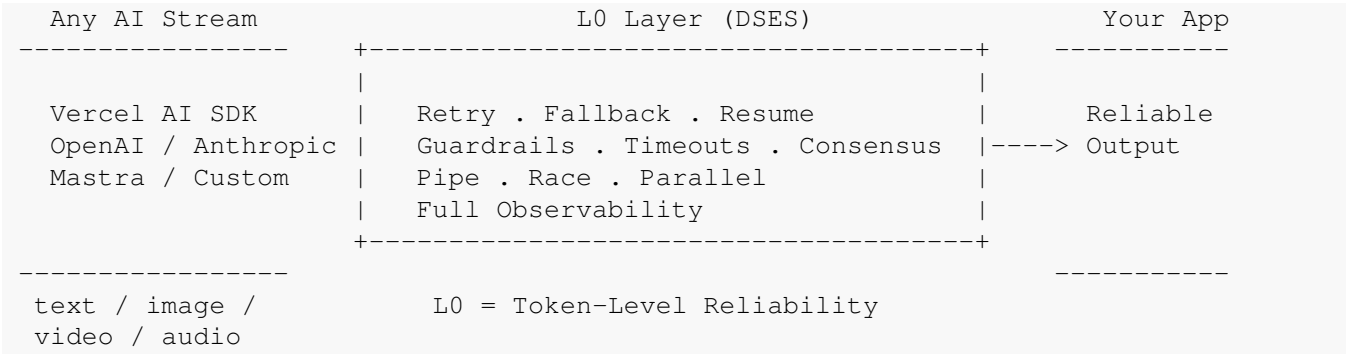
1. **Minimal footprint**

21KB gzipped core. Tree-shakeable with subpath exports (`/core`, `/structured`, `/consensus`, `/parallel`, `/window`, `/drift`, `/monitoring`). No frameworks, no heavy abstractions.

—

System Overview

L0 sits between your application and any streaming model API:



L0's core primitive is `l0()`: you provide a *stream factory*, and L0 returns a normalized event stream plus final state, errors, and telemetry:

```
import { l0, recommendedGuardrails, recommendedRetry } from "reliable-ai-streams";
import { streamText } from "ai";
import { openai } from "@ai-sdk/openai";

const result = await l0({
  stream: () => streamText({ model: openai("gpt-4o"), prompt }),
  fallbackStreams: [() => streamText({ model: openai("gpt-4o-mini"), prompt })],
  guardrails: recommendedGuardrails,
  retry: recommendedRetry,
  timeout: { initialToken: 5000, interToken: 10000 },
  detectDrift: true,
  monitoring: { enabled: true },
  onRetry: (attempt, reason) => console.log(
    XXPROTECTEDBACKTICKXX1XXPROTECTEDBACKTICKXX),
  onFallback: (index, reason) => console.log(
    XXPROTECTEDBACKTICKXX2XXPROTECTEDBACKTICKXX),
});

for await (const event of result.stream) {
  if (event.type === "token") process.stdout.write(event.value);
}

// After stream completes:
console.log(result.state.tokenCount); // total tokens received
console.log(result.state.violations); // guardrail violations
console.log(result.telemetry); // timing, throughput, retries
```

—

Deterministic Lifecycle

L0 defines a deterministic lifecycle state machine for all executions:

```
start -> stream events -> checkpoint/guardrail/drift/timeout hooks
      -> completion or error -> retry/fallback/resume or halt
```

Every transition is observable through lifecycle callbacks:

Callback	Fires when
`onStart`	Execution begins (including retries/fallbacks)
`onToken`	Each text token arrives
`onEvent`	Any normalized event is emitted
`onCheckpoint`	A checkpoint is saved
`onViolation`	A guardrail violation is detected
`onDrift`	Semantic drift is detected
`onTimeout`	A TTFT or inter-token timeout fires
`onRetry`	A retry is initiated (with attempt count and reason)
`onFallback`	A fallback model is activated
`onResume`	Resumption from a checkpoint begins
`onToolCall`	A tool call is detected in the stream
`onAbort`	The execution is cancelled via AbortSignal
`onError`	An error occurs (with willRetry/willFallback flags)
`onComplete`	Execution finishes (with final state)

The lifecycle is specified precisely enough to be implemented identically across the TypeScript and Python runtimes.

—

Normalized Streaming Events and State

L0 normalizes provider-specific events into a unified `L0Event` stream:

```
interface L0Event {
  type: "token" | "message" | "data" | "progress" | "error" | "complete";
  value?: string; // text content (for token events)
  role?: string; // message role
  data?: L0DataPayload; // multimodal content (image, audio, video, file)
  progress?: L0Progress; // progress info for long-running operations
  error?: Error; // error details
  reason?: ErrorCategory; // error category for recovery decisions
  timestamp?: number; // event timestamp
  usage?: { input_tokens: number; output_tokens: number; cost?: number };
}
```

L0 maintains an internal `L0State` throughout execution that tracks:

- accumulated content and token counts,
- checkpoints (last known good content),
- retry counters (model vs network, tracked separately),
- fallback index (0 = primary, 1+ = fallback models),
- guardrail violations (by rule and severity),
- drift detection flags,
- timing (first token timestamp, last token timestamp, total duration),
- categorized network error history,
- multimodal payloads and progress updates.

This state is the basis for all deterministic recovery decisions and is returned alongside the stream for post-run analysis.

—

Reliability Layer

Smart Retries (Model vs Network). Not all failures are equal. L0 categorizes errors into seven categories – `network`, `transient`, `model`, `content`, `provider`, `fatal`, `internal` – and uses that category to decide whether a retry should occur and whether it should count toward configured limits.

The key distinction: **network errors retry with backoff but do not count toward model retry limits**, while guardrail violations and model errors do. This prevents network instability from exhausting your model retry budget.

```
// Retry presets for different risk profiles:
minimalRetry; // 2 attempts, 4 max, linear backoff
recommendedRetry; // 3 attempts, 6 max, fixed-jitter (default)
strictRetry; // 3 attempts, 6 max, full-jitter
exponentialRetry; // 4 attempts, 8 max, exponential
```

L0 supports five backoff strategies (`exponential`, `linear`, `fixed`, `fixed-jitter`, `full-jitter`), custom delay functions, and per-error-type delay configuration.

Timeouts: TTFT and Inter-Token Gaps. Streaming has two critical timeout dimensions:

- **Time-to-first-token (TTFT)**: how long to wait for the stream to begin producing output.
- **Inter-token gap**: how long a silence between tokens is tolerable before treating the stream as stalled.

L0 enforces both independently and treats violations as recoverable transient failures, triggering the retry/fallback chain.

Network Protection. L0 recognizes 12+ streaming/network failure patterns and applies category-correct retry behavior:

- Connection dropped / ECONNRESET / EPIPE
- SSE aborted / partial chunks
- No bytes received / empty response body
- Runtime killed / background throttling (mobile/edge)
- DNS resolution failures
- 429 (rate limit) / 503 (service unavailable) load shedding
- TLS/SSL handshake failures

Each pattern is detected automatically and classified into the appropriate error category, so recovery behavior is correct without manual configuration.

Zero-Token and Stall Protection. A subtle failure mode: the model produces nothing, or produces a few tokens then stops. L0 detects:

- Zero output (stream completes with no meaningful content),
- Early termination (stream closes far sooner than expected),
- Mid-stream stalls (tokens stop arriving but the stream doesn't close).

These are treated as recoverable failures, triggering retry or fallback automatically.

Fallback Models. When retries are exhausted, L0 falls through to a configurable sequence of fallback stream factories. This enables high-availability execution across models and providers while preserving a single deterministic contract to your application:

```
const result = await l0({
  stream: () => streamText({ model: openai("gpt-4o"), prompt }),
  fallbackStreams: [
    () => streamText({ model: openai("gpt-4o-mini"), prompt }),
    () => streamText({ model: anthropic("claude-sonnet-4-20250514"), prompt }),
  ],
});
```

Each fallback gets its own full retry budget. The `onFallback` callback fires on each transition.

—

Guardrails: Streaming-Safe Validation

Guardrails in L0 are **pure validation functions**. They inspect streaming output without rewriting it, returning violations and signaling whether to retry or halt. This is a deliberate design choice: L0 preserves integrity rather than silently patching output.

Built-in Guardrails. L0 provides streaming-structure validators:

- **JSON** (`jsonRule`): streaming-aware structural correctness; tracks brace/bracket depth incrementally. Strict mode enforces parseability and root type.
- **Markdown** (`markdownRule`): validates fences, tables, lists, and detects output that ends mid-sentence.
- **LaTeX** (`latexRule`): validates environments and math delimiters (`$`, `$$`, `\begin /\end `).
- **Pattern** (`patternRule`): detects known bad patterns – meta-commentary ("As an AI..."), instruction leak markers, placeholder text, excessive repetition, and more. Extensible via `customPatternRule`.
- **Zero output** (`zeroOutputRule`): detects empty or meaningless output.

Guardrails are composable and available as presets:

```
minimalGuardrails; // JSON + zero output
recommendedGuardrails; // JSON, Markdown, patterns, zero output
strictGuardrails; // JSON, Markdown, LaTeX, patterns, zero output
jsonOnlyGuardrails; // JSON only
markdownOnlyGuardrails; // Markdown only
```

Fast/Slow Path Execution. To avoid blocking the token loop, guardrails execute on two paths:

- **Fast path** (synchronous): lightweight delta checks that run inline with each token batch. Incremental JSON depth tracking, pattern matching on recent content.
- **Slow path** (asynchronous): heavier full-content scans that run on configurable intervals (default: every 15 tokens) without blocking the stream.

This architecture keeps streaming responsive while still validating correctness.

—

Drift Detection

Even when output is structurally "valid," it can drift in ways that break downstream usage. L0 detects seven drift types:

1. **Tone shift**: output changes register/formality unexpectedly.
2. **Meta-commentary**: model starts commenting on its own output ("As an AI language model...").
3. **Format collapse**: structured output degrades into unstructured prose.
4. **Repetition**: sentences or phrases loop.
5. **Entropy spike**: sudden increase in randomness/incoherence.
6. **Markdown collapse**: structured Markdown degrades.
7. **Hedging spiral**: excessive qualifications and uncertainty markers.

For performance, drift checks operate over a **sliding window** (default 500 characters) rather than rescanning the entire output, keeping cost at O(windowSize) instead of O(contentLength). Drift detection is opt-in (`detectDrift: true`) and can trigger retries when drift is detected.

—

Checkpoints and Last-Known-Good Resumption

If a stream disconnects at token 1500, starting over wastes time and money. L0 supports an opt-in resumption mode that continues from the **last known good checkpoint**.

How it works:

1. L0 periodically saves checkpoints at configurable token intervals (default: every 20 tokens).
2. On retry or fallback, L0 validates the checkpoint content with guardrails and drift detection.
3. If valid, L0 replays the checkpoint content first and optionally builds a continuation prompt to instruct the model to pick up where it left off.

Smart Continuation Deduplication. When models continue from a checkpoint, they often repeat the last few words. L0 includes automatic overlap deduplication (enabled by default when continuation is enabled) that detects and removes repeated suffix/prefix overlap while preserving meaning.

Safety Limitation. Checkpoint continuation is **not recommended for structured JSON output**, because prepending partial JSON can corrupt the structure. L0 enforces this: structured objects are never resumed. In those cases, retry from scratch is the safe default.

—

Structured Output (JSON + Schemas)

For applications that require machine-readable output, L0 provides `structured()` – a dedicated function for schema-validated JSON extraction:

```
import { structured } from "reliable-ai-streams";
import { z } from "zod";

const result = await structured({
  schema: z.object({
    name: z.string(),
    age: z.number(),
    occupation: z.string(),
  }),
  stream: () => streamText({ model: openai("gpt-4o-mini"), prompt }),
  autoCorrect: true, // fix trailing commas, missing braces, markdown fences
});

console.log(result.data); // { name: "Alice", age: 32, occupation: "Engineer" }
```

Supported schema libraries: **Zod v3**, **Zod v4**, **Effect Schema**, and **JSON Schema**.

Auto-correction handles common truncation issues without rewriting semantics:

- Missing closing braces/brackets/quotes
- Trailing commas
- Markdown code fences wrapping JSON
- Duplicate quotes

Additional structured variants: `structuredObject()`, `structuredArray()`, and `structuredStream()` (for streaming validation with a final correctness contract).

—

Multi-Model Patterns: Consensus, Race, Parallel, Pipe

Consensus. For safety-critical or high-confidence tasks, L0 runs multiple independent generations and resolves disagreements:

```
import { consensus } from "reliable-ai-streams";

const result = await consensus({
  streams: [
    () => streamText({ model: openai("gpt-4o"), prompt }),
    () => streamText({ model: anthropic("claude-sonnet-4-20250514"), prompt }),
    () => streamText({ model: openai("gpt-4o-mini"), prompt }),
  ],
  strategy: "majority", // or "unanimous", "weighted", "best"
  conflictResolution: "vote", // or "merge", "fail", "best"
});

console.log(result.confidence); // 0-1 confidence score
console.log(result.agreements); // points of agreement
console.log(result.disagreements); // points of disagreement
```

Consensus strategies: `unanimous` (all must agree), `majority` (most common wins), `weighted` (custom weights), `best` (highest quality). Conflict resolution: `vote` (majority vote), `merge` (merge intelligently), `best` (select highest quality), `fail` (fail on conflict).

Consensus works with both text and structured (schema-based) output, with field-level agreement analysis for structured data. Presets: `strict()`, `standard()`, `lenient()`, `best()`.

Race. Run multiple models in parallel and keep the **first valid result**. Ideal for ultra-low-latency chat and high-availability systems where you want the fastest provider to win:

```
import { race } from "reliable-ai-streams";

const result = await race([
  () => l0({ stream: () => streamText({ model: openai("gpt-4o"), prompt }) }),
  () =>
    l0({
      stream: () =>
        streamText({ model: anthropic("claude-sonnet-4-20250514"), prompt }),
    }),
]);
```

Parallel. Fan-out multiple streams simultaneously with concurrency control, then collect results:

```
import { parallel } from "reliable-ai-streams";

const results = await parallel(
  [
    () => l0({ stream: () => streamText({ model, prompt: prompt1 }) }),
    () => l0({ stream: () => streamText({ model, prompt: prompt2 }) }),
    () => l0({ stream: () => streamText({ model, prompt: prompt3 }) }),
  ],
  { concurrency: 2 },
);
```

Pipe: Streaming Pipelines. Compose multiple streaming steps into a pipeline with safe state passing and guardrails between each stage:

```
import { pipe } from "reliable-ai-streams";

const result = await pipe([
  { stream: () => streamText({ model, prompt: "Summarize this..." }) },
  { stream: (prev) => streamText({ model, prompt:
   XXPROTECTEDBACKTICKXX12XXPROTECTEDBACKTICKXX }) },
  { stream: (prev) => streamText({ model, prompt:
   XXPROTECTEDBACKTICKXX13XXPROTECTEDBACKTICKXX }) },
]);
```

Pipeline presets: `fastPipeline`, `reliablePipeline`, `productionPipeline`.

—

Document Windows

For long documents that exceed model context limits, L0 provides built-in chunking with context preservation:

```
import { createWindow } from "reliable-ai-streams";

const chunks = createWindow(longDocument, {
  strategy: "paragraph", // or "token", "sentence", "character"
  overlap: 100, // overlap for context restoration
});
```

Presets: `smallWindow`, `mediumWindow`, `largeWindow`, `paragraphWindow`, `sentenceWindow`.

—

Event Sourcing and Deterministic Replay

Reliability is only half the story. Debugging and audits demand reproducibility.

L0 includes an event sourcing system that records every stream operation as an atomic, immutable event:

Event Type	Records
`START`	Execution initiated (attempt number, flags)
`TOKEN`	Individual token received
`CHECKPOINT`	Checkpoint saved (content, token count)
`GUARDRAIL`	Guardrail check result (pass/violation)
`DRIFT`	Drift detection result
`RETRY`	Retry initiated (attempt, reason, category)
`FALLBACK`	Fallback activated (index, reason)
`CONTINUATION`	Checkpoint resumption began
`COMPLETE`	Execution finished (final state)
`ERROR`	Error occurred (category, message)

Storage backends include in-memory, file-based, localStorage, TTL-based (auto-expiring), and composite stores. Snapshot support enables faster replay of long executions.

Replay Principle: Ignore External Non-Determinism. In replay mode, L0 does **no network calls**, performs **no retries**, and does **no recomputation** of guardrails or drift. It rehydrates the exact recorded events, producing deterministic reproduction. Lifecycle callbacks still fire during replay (for testing and debugging), but no side effects occur.

Replay supports configurable playback speed, partial replay via sequence ranges (`fromSeq`/`toSeq`), and comparison between replays for consistency verification.

This design makes failures reproducible in tests and enables production-grade audit trails for compliance.

—

Monitoring and Telemetry

L0 ships with built-in observability integrations:

- **OpenTelemetry**: spans, metrics, and traces for every execution phase.
- **Sentry**: automatic error tracking and breadcrumbs.
- **Custom event handlers**: subscribe to categorized lifecycle events.

Telemetry tracks:

- throughput (tokens/sec), total duration, token counts,
- TTFT and inter-token timing distributions,
- retry attempts (network vs model, separately),
- guardrail violations by rule, severity, and frequency,
- drift events by type,
- network error types and frequencies,
- continuation usage and checkpoint metrics,
- fallback transitions and model usage.

Telemetry is returned alongside the result object in `result.telemetry`, enabling straightforward logging, dashboards, alerts, and trace correlation. Sampling is configurable for high-throughput production use.

—

Adapters: Bring Your Own Provider

L0 ships with built-in adapters for major LLM SDKs:

Adapter	Import	Usage
Vercel AI	Native (no adapter needed)	`stream: () => streamText( model, prompt )`
OpenAI	`openaiStream` from `reliable-ai-streams`	`stream: openaiStream(client, params)`
Anthropic	`anthropicStream` from `reliable-ai-streams`	`stream: anthropicStream(client, params)`
Mastra	`mastraStream` from `reliable-ai-streams`	`stream: mastraStream(agent, prompt)`

For custom providers, L0 provides an adapter registry and helper functions (`toL0Events()`, `createAdapterTokenEvent()`, `createAdapterDoneEvent()`) to convert any async iterable into L0's normalized event format.

—

Tool Call Support

L0 provides first-class support for streaming tool calls (function calling):

- Detects tool call events as they stream and buffers arguments incrementally.
- Emits tool call events with the tool name, call ID, and accumulated arguments.
- Reports tool calls via the `onToolCall` lifecycle callback.
- Tracks all detected tool calls in the execution state.
- Includes formatting utilities (`formatTool`, `formatTools`) supporting JSON Schema, TypeScript, Natural Language, and XML output styles.
- Provides `parseFunctionCall()` for parsing function calls from model output.

This enables agent runtimes to react to tool calls in real time while still benefiting from L0's full reliability stack.

—

Multimodal Support

L0's event model extends beyond text. The `L0DataPayload` type supports:

- **Content types**: `text`, `image`, `audio`, `video`, `file`, `json`, `binary`
- **Metadata**: width, height, duration, file size, filename, seed, model info
- **Progress tracking**: percentage, stage, and ETA for long-running generation

This enables building adapters for image generation (FLUX.2, Stable Diffusion), video generation (Veo 3), audio generation (CSM), and other multimodal models – all under the same deterministic lifecycle, with the same retry, fallback, and observability guarantees.

—

JSON Auto-Healing and Format Repair

LLM output frequently arrives with structural defects. L0 provides automatic repair utilities:

- **JSON**: missing closing braces/brackets/quotes, trailing commas, duplicate quotes, extraction from surrounding prose or Markdown fences.
- **Markdown**: unterminated code fences, broken table formatting.
- **LaTeX**: unclosed environments.
- **Tool calls**: malformed function call arguments.

These repairs are applied only when explicitly enabled (`autoCorrect: true`) and are tracked – `result.corrections` reports exactly what was fixed, so repairs are never silent.

—

Performance

L0 is designed to stay far ahead of model inference speeds, even with the full feature stack enabled.

Scenario	Tokens/s	Avg Duration	TTFT	Overhead
Baseline (raw streaming)	4,459,021	0.45 ms	0.00 ms	–
L0 Core (no features)	1,068,683	1.87 ms	0.04 ms	316
L0 + JSON Guardrail	615,031	3.28 ms	0.20 ms	629
L0 + All Guardrails	337,174	5.93 ms	0.08 ms	1218
L0 + Drift Detection	609,546	3.37 ms	0.07 ms	649
L0 Full Stack	**259,478**	**7.73 ms**	**0.08 ms**	**1618

Benchmark Results (Apple M1 Max, Node.js 24, zero-delay mock streams).

Key Optimizations.

- **Incremental JSON state tracking**: O(delta) per token instead of O(content). Full content scans only at stream completion.
- **Sliding-window drift detection**: checks operate on a configurable window (default 500 chars) instead of full content.
- **Tunable check intervals**: guardrails every 15 tokens, drift every 25 tokens, checkpoints every 20 tokens (all configurable).
- **Fast/slow path guardrails**: synchronous delta checks inline; heavy scans deferred to async intervals.

Inference Speed Headroom. Even with the full stack enabled, L0 sustains ~260K tokens/s – orders of magnitude above current and next-generation inference hardware:

GPU Generation	Expected Tokens/s	L0 Headroom
Current (H100)	~100-200	1,300-2,600x
Blackwell (B200)	~1,000+	~260x

The substrate will not be the bottleneck.

Export	Minified	Gzipped
`reliable-ai-streams`	191 KB	56 KB
`reliable-ai-streams/core`	71 KB	21 KB
`reliable-ai-streams/structured`	61 KB	18 KB
`reliable-ai-streams/consensus`	72 KB	21 KB
`reliable-ai-streams/parallel`	58 KB	17 KB
`reliable-ai-streams/window`	62 KB	18 KB
`reliable-ai-streams/guardrails`	18 KB	6 KB
`reliable-ai-streams/drift`	4 KB	2 KB
`reliable-ai-streams/monitoring`	27 KB	7 KB

Bundle Size. Tree-shakeable with subpath exports. No frameworks. No heavy abstractions.

—

What "Deterministic" Means Here

L0 is not claiming the model is deterministic. It's claiming the *execution substrate* is:

- The lifecycle state machine is specified and identical across implementations.
- Events are normalized into a consistent shape regardless of provider.
- State tracking is consistent and observable at every point.
- Recovery decisions (retry, fallback, halt) are rule-driven, category-aware, and auditable.
- Full executions can be recorded and replayed byte-for-byte from the event log.

That's enough determinism to make token streams reliable.

—

Testing

L0 is validated by 3,000+ unit tests and 250+ integration tests covering:

- All guardrail rules (JSON, Markdown, LaTeX, pattern, zero output)
- Drift detection (all seven drift types, sliding window behavior)
- Retry logic (all backoff strategies, error categories, budget tracking)
- Network error detection (all 12+ failure patterns)
- Structured output (Zod v3/v4, Effect Schema, JSON Schema, auto-correction)
- Consensus (all strategies and conflict resolution modes)
- Parallel, race, and pipe operations
- Event sourcing (recording, replay, snapshots, storage backends)
- Adapters (OpenAI, Anthropic, Mastra, Vercel AI, custom)
- Checkpoint resumption and continuation deduplication
- Timeout enforcement (TTFT and inter-token)

—

Use Cases

- **Production chat**: consistent streaming semantics, timeouts, retries, fallbacks, and telemetry for user-facing applications.
- **Agent orchestration**: tool calls, partial failures, and multi-step reasoning with deterministic recovery at every step.
- **Structured extraction**: guaranteed-valid JSON with schema enforcement, auto-correction, and retry on validation failure.
- **Compliance and supervision**: guardrails, drift detection, and audit-ready replay logs for regulated industries.
- **Low-latency pipelines**: race for fastest-provider-wins, parallel for fan-out/fan-in, pipe for multi-stage streaming.
- **High-confidence generation**: multi-model consensus for safety-critical tasks where a single model's output is insufficient.
- **Multimodal applications**: image, audio, and video generation with the same reliability guarantees as text.
- **Long document processing**: document windowing with overlap for context-preserving chunking.

—

Appendix A: Error Taxonomy

Code	Category	Description
`STREAM_ABORTED`	transient	Stream was aborted unexpectedly
`INITIAL_TOKEN_TIMEOUT`	transient	No first token within deadline
`INTER_TOKEN_TIMEOUT`	transient	Gap between tokens exceeded limit
`ZERO_OUTPUT`	content	Model produced empty/trivial output
`GUARDRAIL_VIOLATION`	content	Output violated a guardrail rule
`FATAL_GUARDRAIL_VIOLATION`	fatal	Output violated a fatal guardrail
`DRIFT_DETECTED`	content	Semantic drift detected
`INVALID_STREAM`	fatal	Stream is not a valid async iterable
`ADAPTER_NOT_FOUND`	fatal	No adapter could handle the stream
`ALL_STREAMS_EXHAUSTED`	fatal	All retries and fallbacks failed
`NETWORK_ERROR`	network	Transport-level failure
`FEATURE_NOT_ENABLED`	fatal	Required feature not enabled

Error Codes.

Category	Recovery Behavior	Counts Toward Model Limit?
`network`	Retry with backoff	No
`transient`	Retry with backoff	No
`model`	Retry with limits, may trigger fallback	Yes
`content`	Retry with limits (guardrail/drift violation)	Yes
`provider`	Retry with limits, may trigger fallback	Yes
`fatal`	Halt immediately (auth failure, invalid config)	N/A
`internal`	Halt immediately (bug in L0 itself)	N/A

Error Categories. —

Appendix B: Drift Types

Type	Detection Method
`tone_shift`	Register/voice change analysis
`meta_commentary`	AI self-reference pattern matching
`format_collapse`	Structural degradation detection
`markdown_collapse`	Markdown formatting breakdown
`repetition`	Phrase/sentence loop detection
`entropy_spike`	Statistical surprise in token distribution
`hedging`	Excessive qualification language

—

Appendix C: Guardrail Severity

Violations carry severity which influences L0's recovery decision:

Severity	Behavior
`warning`	Recorded but execution continues
`error`	Triggers retry (counts toward model retry budget)
`fatal`	Halts execution immediately

—

Appendix D: Feature Opt-In Model

Heavy features use explicit enablement for tree-shaking:

```
import { enableDriftDetection, DriftDetector } from "reliable-ai-streams";
import { enableMonitoring, L0Monitor } from "reliable-ai-streams";
import { enableInterceptors, InterceptorManager } from "reliable-ai-streams";
import { enableAdapterRegistry } from "reliable-ai-streams";

enableDriftDetection(() => new DriftDetector());
enableMonitoring((config) => new L0Monitor(config));
enableInterceptors((list) => new InterceptorManager(list));
enableAdapterRegistry(registry);
```

This ensures unused features are excluded from production bundles.

—

Appendix E: Event Sourcing Event Types

`START` · `TOKEN` · `CHECKPOINT` · `GUARDRAIL` · `DRIFT` · `RETRY` · `FALLBACK` · `CONTINUATION` · `COMPLETE` · `ERROR`

Each event is timestamped and carries a type-specific payload sufficient for exact replay.

ABOUT THIS MANUSCRIPT

This manuscript was prepared using Rχiv-Maker v1.22.0 (1).

EXTENDED AUTHOR INFORMATION

Bibliography

1. Bruno M. Saraiva, António D. Brito, Guillaume Jaquemet, and Ricardo Henriques. Rxiv-maker: an automated template engine for streamlined scientific publications, 2025. doi:10.48550/arXiv.2508.00836.